1. Write a OpenMP program to sort an array on n elements using both sequential and parallel mergesort(using Section). Record the difference in execution time.

What is OpenMp?

OpenMP is a set of compiler directives as well as an API for programs written in C, C++, or FORTRAN that provides support for parallel programming in shared- memory environments. OpenMP identifies parallel regions as blocks of code that may run in parallel. Application developers insert compiler directives into their code at parallel regions, and these directives instruct the OpenMP run-time library to execute the region in parallel.

What is merge sort?

Merge sort is a divide and conquer algorithm that sorts the input array by breaking it into sub arrays until they have 1 element, and then merging the results back into a sorted array. The time and space complexities are O(N log N) and O(n) respectively. Sequential Merge Sort

Sequential merge sort in C is a sorting algorithm that follows the divide-and-conquer paradigm. It recursively divides an array into two halves until the sub-arrays contain only one element (which is considered sorted). Then, it merges these sub-arrays in a sorted manner to produce new sorted sub-arrays, repeating this process until the entire array is sorted.

Parallel Merge Sort

Parallel merge sort is an algorithm that sorts a list by recursively dividing it into sublists, sorting them independently using multiple threads or processes, and then merging the sorted sublists. This approach leverages parallel processing to improve sorting speed, especially for large datasets.Section:

Sections are independent blocks of code, able to be assigned to separate threads if they are available.

Code:

```
#include<stdio.h>
#include <stdlib.h>
#include <omp.h>
void merge(int arr[], int left, int mid, int right) {
int n1 = mid - left + 1;
int n2 = right - mid;
int L[n1], R[n2];
for (int i = 0; i < n1; i++)
L[i] = arr[left + i];
for (int j = 0; j < n2; j++)
R[j] = arr[mid + 1 + j];
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
if (L[i] <= R[j])
arr[k++] = L[i++];
else
arr[k++] = R[j++];
```

```
}
while (i < n1)
arr[k++] = L[i++];
while (j < n2)
arr[k++] = R[j++];
}
// Sequential Merge Sort
void mergeSortSequential(int arr[], int left, int right) {
if (left < right) {
int mid = left + (right - left) / 2;
mergeSortSequential(arr, left, mid);
mergeSortSequential(arr, mid + 1, right);
merge(arr, left, mid, right);
}
}
// Parallel Merge Sort
void mergeSortParallel(int arr[], int left, int right) {
if (left < right) {
int mid = left + (right - left) / 2;
#pragma omp parallel sections
{
#pragma omp section
mergeSortParallel(arr, left, mid);
#pragma omp section
mergeSortParallel(arr, mid + 1, right);
}
merge(arr, left, mid, right);
}
}
int main() {
int n;
printf("Enter number of elements: ");
scanf("%d", &n);
int *arr1 = (int *)malloc(n * sizeof(int));
int *arr2 = (int *)malloc(n * sizeof(int));
printf("Enter %d elements: ", n);
for (int i = 0; i < n; i++) {
scanf("%d", &arr1[i]);
arr2[i] = arr1[i]; // Copy for parallel sorting
}
double start, end;
// Sequential Sort
start = omp_get_wtime();
mergeSortSequential(arr1, 0, n - 1);
end = omp_get_wtime();
printf("Sequential Merge Sort Time: %f seconds\n",end-start);
// Parallel Sort
```

```
start = omp_get_wtime();
mergeSortParallel(arr2, 0, n - 1);
end = omp_get_wtime();

printf("Parallel Merge Sort Time: %f seconds\n",end-start);
printf("Sortedarray: ");
for (int i = 0; i < n; i++)
printf("%d ", arr1[i]);
printf("\n");
free(arr1);
free(arr2);
return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$gcc-fopenmp mergesort.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
Enter number of elements: 6
Enter 6 elements: 2 9 1 5 10 4
Sequential Merge Sort Time: 0.000001 seconds
Parallel Merge Sort Time:
0.000244 seconds Sorted array: 1
2 4 5 9 10
```

2.Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following:
a. Thread 0: Iterations 0 – 1 b. Thread 1: Iterations 2 – 3

What is Scheduling in OpenMP?
Scheduling is a method in OpenMP to distribute iterations to different threads in for loop. Describes how iterations of the loop are divided among the threads in the team. The default schedule is implementation dependent.
Static:
Loop iterations are divided into pieces of size chunk and then statically assigned to threads. If chunk is not specified, the iterations are evenly (if possible) divided contiguously among the threads. If you do not specify chunk-size variable, OpenMP will divides iterations into chunks that are approximately equal in size and it distributes chunks to threads **in order.**

Code:

```
#include <stdio.h>
#include <omp.h>
int main() {
int num_iterations;
printf("Enter the number of iterations: ");
scanf("%d", &num_iterations);
#pragma omp parallel
{
#pragma omp for schedule(static,2)
for (int i = 0; i < num_iterations; i++) {
 printf("Thread %d: Iteration %d\n", omp_get_thread_num(), i);

  }
  }
  return 0;
  }
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc -fopenmp iterations.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
Enter the number of iterations: 6
Thread 1: Iteration 2
Thread 1: Iteration 3
Thread 2: Iteration 4
Thread 2: Iteration 5
Thread 0: Iteration 0
Thread 0: Iteration 1
```

3.Write a OpenMP program to calculate n Fibonacci numbers using tasks.

Tasks:

An OpenMP task is a single line of code or a structured block which is immediately"written down"in a list of tasks. The new task can be executed immediately, or it canbe deferred. If the if clause is used and the argument evaluates to 0, then the task isexecuted immediately, superseding whatever else that thread is doing. There has tobe an existing parallel thread team for this to work. Otherwise one thread ends updoing all tasks and you don't get any contribution to parallelism.

Fibonacci sequence:

The Fibonacci sequence is a sequence where the next term is the sum of the previoustwo terms. The first two terms of the Fibonacci sequence are 0 followed by 1. Ex:Fibonacci Sequence is : 0,1,1,2,3,5,8,13…….

Code:

```
#include <stdio.h>
#include <omp.h>
int fib(int n)
{
int i, j;
if (n<2)
return n;
else
{
#pragma omp task shared(i) firstprivate(n)
i=fib(n-1);
#pragma omp task shared(j) firstprivate(n)
j=fib(n-2);
#pragma omp taskwait
return i+j;
}
}

int main()
{
int n = 10;
omp_set_dynamic(0);
omp_set_num_threads(4);
#pragma omp parallel shared(n)
{
#pragma omp single
printf ("fib(%d) = %d\n", n, fib(n));
}
}
```

OUTPUT:

```
bantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc -fopenmp fibonacci.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
 fib(10) = 55
```

4 .Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.

Parallel Directive:

The omp parallel directive explicitly instructs the compiler to parallelize the chosen block of code.

For Directive:

Causes the work done in a for loop inside a parallel region to be divided among threads.

Prime Number:

A prime number is a positive integer that is divisible only by 1 and itself. For example: 2, 3, 5, 7, 11, 13, 17…..

Code:

1)

```
#include <stdio.h> #include <omp.h> void main()
{
int prime[1000], i, j, n;

// Prompt user for input
printf("\n In order to find prime numbers from 1 to n, enter the value of n:"); scanf("%d", &n);

// Initialize all numbers as prime (set all to 1) for(i = 1; i <= n; i++) {
prime[i] = 1;
        }

// 1 is not a prime number prime[1] = 0;
// Sieve of Eratosthenes with parallelization
for(i = 2; i * i <= n; i++) {
#pragma omp parallel for
for(j = i * i; j <= n; j = j + i) {
 if(prime[j] == 1) {
prime[j] = 0;
}
}
}

// Print prime numbers
printf("\n Prime numbers from 1 to %d are\n", n);
 for(i = 2; i <= n; i++) {
if(prime[i] == 1) {
printf("%d\t", i);
}
}
```

```
printf("\n");
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc -fopenmp prime.c ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
In order to find prime numbers from 1 to n, enter the value of n: 100 Prime numbers from 1 to 100 are
2  3       5       7       11      13      17      19      23      29      31
37 41      43      47      53      59      61      67      71      73
79 83      89      97
```

2)

```
#include <stdio.h>
 #include <omp.h>
int main()
{
int prime[1000], i, j, n;

// Prompt user for input
printf("\n In order to find prime numbers from 1 to n, enter the value of n:"); scanf("%d", &n);

// Initialize all numbers as prime (set all to 1)
for(i = 1; i <= n; i++) {
prime[i] = 1;
}

// 1 is not a prime number
prime[1] = 0;
// Sieve of Eratosthenes with parallelization
for(i = 2; i * i <= n; i++) {
#pragma omp parallel for for
(j = i * i; j <= n; j = j + i) {
if(prime[j] == 1) {
prime[j] = 0;
}
}
}
```

```
// Print prime numbers
printf("\n Prime numbers from 1 to %d are\n", n);
for(i = 2; i <= n; i++) {
if(prime[i] == 1) {
printf("%d\t", i);
}

}
double start, end;
// Sequential
start = omp_get_wtime();
 prime[i];
end = omp_get_wtime();
printf("Sequential Time: %f seconds\n",end-start);
// Parallel
start = omp_get_wtime();
 prime[i];
end = omp_get_wtime();
printf("Parallel Time: %f seconds\n",end-start);
 printf("\n");
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ gcc -fopenmp prime.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
In order to find prime numbers from 1 to n, enter the value of n:50
Prime numbers from 1 to 50 are
2  3        5        7        11       13       17       19       23       29       31
37 41       43       47
Sequential Time: 0.000001 seconds Parallel Time: 0.000000 seconds
```

5.Write a MPI Program to demonstration of MPI_Send and MPI_Recv.Message-

Passing Interface (MPI)

Message-Passing is a communication model used on distributed-memory architecture. MPI is a standard that specifies the message-passing libraries supporting parallel programming in C/C++ or Fortran. MPI_Send and MPI_Recv are the basicbuilding blocks for essentially all of the more specialized MPI commands in MPI. They are also the basic communication tools in MPI application. Since MPI_Send and MPI_Recv involve two ranks, they are called “point-to-point” communication. The process of communicating data follows a standard pattern. Rank A decides to send data to rank B. It first packs the data into a buffer. This avoids sending multiple messages, which would take more time. Rank A then calls MPI_Send to create a message for rank B. The communication device is then given the responsibility of routing the message to the correct destination. Rank B must know that it is about to receive a message and acknowledge this by calling MPI_Recv. MPI_RANK:

"Rank" refers to a unique identifier assigned to each process within a communication group. It's a logical way of numbering processes to facilitate communication between them. MPI automatically assigns ranks in the MPI_COMM_WORLD group when the MPI environment is initialized (MPI_Init).

Code:

1)

```
#include <stdio.h>
 #include "mpi.h"
int main(int argc, char** argv)
{
int my_rank, numbertoreceive [10], numbertosend [3]={73, 2, -16};
 int recv_count, i;
MPI_Status status;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
 if (my_rank==0){
MPI_Recv( numbertoreceive, 3, MPI_INT, MPI_ANY_SOURCE,
MPI_ANY_TAG, MPI_COMM_WORLD,
&status);
printf("status.MPI_SOURCE = %d\n", status.MPI_SOURCE);
 printf("status.MPI_TAG = %d\n", status.MPI_TAG);
printf("status.MPI_ERROR = %d\n", status.MPI_ERROR);
MPI_Get_count(&status, MPI_INT, &recv_count);
printf("Receive %d data\n", recv_count);
for(i = 0; i < recv_count; i++)
printf("recv[%d] = %d\n", i, numbertoreceive[i]);
}
else MPI_Send( numbertosend, 3, MPI_INT, 0, 10, MPI_COMM_WORLD);
MPI_Finalize();
return 0;
```

```
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o send SendRcv.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./send status.MPI_SOURCE =
3
status.MPI_TAG = 10
status.MPI_ERROR = 110


Receive 3 data
recv[0] = 73
recv[1] = 2
recv[2] = -16
```

2)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char** argv) {
// Initialize the MPI environment
 MPI_Init(NULL, NULL);
// Find out rank, size int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
 int world_size;
MPI_Comm_size(MPI_COMM_WORLD, &world_size);
// We are assuming at least 2 processes for this task
if (world_size < 2) {
fprintf(stderr, "World size must be greater than 1 for %s\n", argv[0]);
 MPI_Abort(MPI_COMM_WORLD, 1);
}

int number;
if (world_rank == 0) {
// If we are rank 0, set the number to -1 and send it to process 1
 number = -1;
MPI_Send(
/* data      = */ &number,
/* count    = */ 1,

/* datatype          = */ MPI_INT,
```

```
/* destination = */ 1,
/* tag       = */ 0,
/* communicator = */ MPI_COMM_WORLD);
} else if (world_rank == 1) { MPI_Recv(
/* data     = */ &number,

/* count   = */ 1,
/* datatype          = */ MPI_INT,
/* source  = */ 0,
/* tag       = */ 0,
/* communicator = */ MPI_COMM_WORLD,
/* status   = */ MPI_STATUS_IGNORE);
printf("Process 1 received number %d from process 0\n", number);
}
MPI_Finalize();
}
```

OUTPUT :

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o send sendrcv.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./
send Process -1 received number -1from process -0
```

6.Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.

Deadlock:

Deadlock is an often-encountered situation in parallel processing. It results when two or more processes are in contention for the same set of resources. In communications, a typical scenario involves two processes wishing to exchange messages: each is trying to give a message to the other, but neither of them is ready to accept a message. **What is point-to-point communication in MPI?**

MPI processes communicate by explicitly sending and receiving messages. In point- to-point, messages are sent between two processes. Since MPI processes are independent, in order to coordinate work, they need to communicate by explicitly sending and receiving messages.

Demonstration of deadlock

Code:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
int rank, size, data = 0;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
 MPI_Comm_size(MPI_COMM_WORLD, &size);

if (size < 2) {
printf("This program requires at least 2 processes.\n");
MPI_Abort(MPI_COMM_WORLD, 1);
}

if (rank == 0) {
// Rank 0 sends first, then receives
MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
printf("Process 0 sent data to Process 1\n");

MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
 MPI_STATUS_IGNORE);
printf("Process 0 received data from Process 1\n");
} else if (rank == 1) {
// Rank 1 sends first, then receives
MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
printf("Process 1 sent data to Process 0\n");
MPI_Recv(&data, 1, MPI_INT, 0, 0,
```

```
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
printf("Process 1 received data from Process 0\n");

}
MPI_Finalize(); return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI Mpi7.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./MPI
Process 0 sent data to Process 1
Process 0 received data from Process 1
 Process 1 sent data to Process 0
 Process 1 received data from Process 0
```

Deadlock avoidance

To fix deadlock, need to coordinate the communication between pairs of processes so that there is an ordering of sends and receives between them.

Code:

```
#include <mpi.h>
 #include <stdio.h>

int main(int argc, char** argv) {
 int rank, size, data = 0;
MPI_Init(&argc, &argv);
 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
 MPI_Comm_size(MPI_COMM_WORLD, &size);

if (size < 2) {
printf("This program requires at least 2 processes.\n");
MPI_Abort(MPI_COMM_WORLD, 1);
}
if (rank == 0) {
// Rank 0 sends first
MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
printf("Process 0 sent data to Process 1\n");

MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
```

```
printf("Process 0 received data from Process 1\n");
} else if (rank == 1) {
// Rank 1 receives first
MPI_Recv(&data, 1, MPI_INT, 0, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
printf("Process 1 received data from Process 0\n");

MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
printf("Process 1 sent data to Process 0\n");
}

MPI_Finalize(); return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI avoiddeadlock.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 2 ./MPI
 Process 0 sent data to Process 1
Process 0 received data from Process 1
Process 1 received data from Process 0
Process 1 sent data to Process 0
```

7.Write a MPI Program to demonstration of Broadcast operation.

Broadcasting with MPI_Bcast:

A broadcast is one of the standard collective communication techniques. During a broadcast, one process sends the same data to all processes in a communicator. One of the main uses of broadcasting is to send out user input to a parallel program, or send out configuration parameters to all processes. Basically process zero is the root process, and it has the initial copy of data. All of the other processes receive the copy of data.

1)

Code:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
int rank, size;
int data; // The data to broadcast

MPI_Init(&argc, &argv);          // Initialize MPI
MPI_Comm_rank(MPI_COMM_WORLD, &rank);    // Get process rank
MPI_Comm_size(MPI_COMM_WORLD, &size);      // Get total number of
processes
if (rank == 0) {
data = 42; // Root process sets the data
printf("Process %d is broadcasting data = %d\n", rank, data);
}
// Broadcast the data from process 0 to all other processes
MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD
// All processes print the received data
printf("Process %d received data = %d\n", rank, data);

MPI_Finalize(); // Finalize MPI return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI broadcast.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 2 ./MPI
Process 0 is broadcasting data = 42
Process 0 received data = 42
Process 1 received data = 42
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./MPI
Process 0 is broadcasting data = 42
Process 0 received data = 42
Process 1 received data = 42
Process 2 received data = 42
```

Process 3 received data = 42
Process 4 received data = 42

2)

```
#include <mpi.h>
 #include <stdio.h>
#include <stdlib.h>
#include <assert.h>

#define CONDUCTOR 0

int main(int argc, char** argv) {
int answer = 0;
int numProcs = 0, myRank = 0;

MPI_Init(&argc, &argv);
 MPI_Comm_size(MPI_COMM_WORLD, &numProcs);
 MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
if (myRank == CONDUCTOR) {
 answer = 42;
}
printf("BEFORE the broadcast, process %d's answer = %d\n", myRank, answer);
MPI_Bcast(&answer, 1, MPI_INT, 0, MPI_COMM_WORLD);
printf("AFTER the broadcast, process %d's answer = %d\n",myRank, answer);
MPI_Finalize();
return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI Broadcast.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./MPI
BEFORE the broadcast, process 0's answer = 42
AFTER the broadcast, process 0's answer = 42
BEFORE the broadcast, process 1's answer = 0
AFTER the broadcast, process 1's answer = 42
BEFORE the broadcast, process 3's answer = 0
BEFORE the broadcast, process 2's answer = 0
AFTER the broadcast, process 2's answer = 42
BEFORE the broadcast, process 4's answer = 0
AFTER the broadcast, process 3's answer = 42
AFTER the broadcast, process 4's answer = 42
```

8.Write a MPI Program demonstration of MPI_Scatter and MPI_Gather

MPI_Scatter:

MPI_Scatter is a collective routine that is very similar to MPI_Bcast. MPI_Scatter involves a designated root process sending data to all processes in a communicator. The primary difference between MPI_Bcast and MPI_Scatter is small but important. MPI_Bcast sends the same piece of data to all processes while MPI_Scatter sends chunks of an array to different processes.

MPI_Gather:

MPI_Gather is the inverse of MPI_Scatter. Instead of spreading elements from one process to many processes, MPI_Gather takes elements from many processes and gathers them to one single process. This routine is highly useful to many parallel algorithms, such as parallel sorting and searching.

Code:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
int size, rank;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

int globaldata[4]; /*wants to declare array this way*/
int localdata[4]; /*without using pointers*/
int i;
if (rank == 0) {
   for (i = 0; i < size; i++)
   globaldata[i] = i;
      printf("1. Processor %d has data: ", rank);
   for (i = 0; i < size; i++)
   printf("%d ", globaldata[i]);
   printf("\n");
}
MPI_Scatter(globaldata,1,MPI_INT,&localdata,1,MPI_INT,0, MPI_COMM_WORLD);
printf("2. Processor %d has data %d\n", rank, localdata[rank]); localdata[rank]= 5;
printf("3. Processor %d now has %d\n", rank, localdata[rank]);
MPI_Gather(&localdata,1,MPI_INT,globaldata,1,MPI_INT,0, MPI_COMM_WORLD);
if (rank == 0) {
   printf("4. Processor %d has data: ", rank); for (i = 0; i < size; i++)
   printf("%d ", globaldata[i]); printf("\n");
}

MPI_Finalize(); return 0;
  }
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI scattergather.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./MPI
1. Processor 0 has data: 0 1 2 3 4
2. Processor 0 has data 0
3. Processor 0 now has 5
2. Processor 1 has data 0
3. Processor 1 now has 5
2. Processor 2 has data 0
3. Processor 2 now has 5
2. Processor 3 has data 0
3. Processor 3 now has 5
2. Processor 4 has data 255
3. Processor 4 now has 5
4. Processor 0 has data: 5 1 2 3 4
```

2)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

 int main(int argc, char **argv) {
 int size, rank;
 MPI_Init(&argc, &argv);
 MPI_Comm_size(MPI_COMM_WORLD, &size);
 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  int globaldata[4];/*wants to declare array this way*/
  int localdata;/*without using pointers*/

int i;
if (rank == 0) {
for (i=0; i<size; i++) globaldata[i] = i;
printf("1. Processor %d has data: ", rank); for (i=0; i<size; i++)
printf("%d ", globaldata[i]); printf("\n");
}
MPI_Scatter(globaldata,1,MPI_INT,&localdata,1,MPI_INT,0,
MPI_COMM_WORLD);

printf("2. Processor %d has data %d\n", rank, localdata);
localdata= 5;
printf("3. Processor %d now has %d\n", rank, localdata);

MPI_Gather(&localdata,1,MPI_INT,globaldata,1,MPI_INT,0,
 MPI_COMM_WORLD);
if (rank == 0) {
printf("4. Processor %d has data: ", rank);
```

```
 for (i=0; i<size; i++)
 printf("%d ", globaldata[i]);

 printf("\n");
    }
    MPI_Finalize();
     return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI scattergather.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./MPI
1. Processor 0 has data: 0 1 2 3 4
2. Processor 0 has data 0
3. Processor 0 now has 5
2. Processor 1 has data 1
3. Processor 1 now has 5
2. Processor 2 has data 2
3. Processor 2 now has 5
2. Processor 3 has data 3
3. Processor 3 now has 5
2. Processor 4 has data 4
3. Processor 4 now has 5
Processor 0 has data: 5 5 5 5 5
```

9.Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)

MPI_Reduce:

Reduce is a classic concept from functional programming. Data reduction involves reducing a set of numbers into a smaller set of numbers via a function. For example, let's say we have a list of numbers [1, 2, 3, 4, 5]. Reducing this list of numbers with the sum function would produce sum([1, 2, 3, 4, 5]) = 15. Similarly, the multiplication
reduction would yield multiply([1, 2, 3, 4, 5]) = 120. MPI_Reduce takes an array of input elements on each process and returns an array of output elements to the root
process. The output elements contain the reduced result.

MPI_AllReduce:

Many parallel applications will require accessing the reduced results across all processes rather than the root process. In a similar complementary style of MPI_Allgather to MPI_Gather, MPI_Allreduce will reduce the values and distribute the results to all processes.

Code:

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
int rank, size;
int value, sum, product, max, min;
int all_sum, all_product, all_max, all_min;

MPI_Init(&argc, &argv);// Initialize
MPI MPI_Comm_rank(MPI_COMM_WORLD, &rank);// Get rank
MPI_Comm_size(MPI_COMM_WORLD, &size);  // Get number of processes

value = rank + 1;  // Each process has a unique value

// ---- MPI_Reduce ----
MPI_Reduce(&value, &sum,1, MPI_INT, MPI_SUM,  0, MPI_COMM_WORLD);
MPI_Reduce(&value, &product,  1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
MPI_Reduce(&value, &max,1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
MPI_Reduce(&value, &min,1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);

if (rank == 0) {
printf("=== MPI_Reduce results at root ===\n");
printf("SUM     = %d\n", sum);
printf("PRODUCT = %d\n", product);
printf("MAX     = %d\n", max);
printf("MIN     = %d\n", min);
}

// ---- MPI_Allreduce ----
MPI_Allreduce(&value, &all_sum,1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

```
MPI_Allreduce(&value, &all_product, 1, MPI_INT, MPI_PROD, MPI_COMM_WORLD);
MPI_Allreduce(&value, &all_max,1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
MPI_Allreduce(&value, &all_min,1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

printf("Process %d: ALL_SUM = %d, ALL_PROD = %d, ALL_MAX = %d,
ALL_MIN = %d\n",
rank, all_sum, all_product, all_max, all_min);

MPI_Finalize();  // Finalize MPI return 0;
}
```

OUTPUT:

```
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpicc -g -o MPI reduceallreduce.c
ubantu@ubantu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mpirun -np 5 ./MPI
=== MPI_Reduce results at root === SUM = 15
PRODUCT = 120
MAX    = 5
MIN    = 1
Process 0: ALL_SUM = 15, ALL_PROD = 120, ALL_MAX = 5, ALL_MIN = 1
Process 1: ALL_SUM = 15, ALL_PROD = 120, ALL_MAX = 5, ALL_MIN = 1
Process 2: ALL_SUM = 15, ALL_PROD = 120, ALL_MAX = 5, ALL_MIN = 1
Process 3: ALL_SUM = 15, ALL_PROD = 120, ALL_MAX = 5, ALL_MIN = 1
Process 4: ALL_SUM = 15, ALL_PROD = 120, ALL_MAX = 5, ALL_MIN = 1
```

VIVA QUESTIONS

1. OpenMP – Sequential vs Parallel Merge Sort
    1. What is merge sort, and why is it suitable for parallelization?
    2. How does OpenMP's section directive help in parallelizing merge sort?
    3. What is the role of recursion in parallel merge sort?
    4. How do you measure execution time in OpenMP?
    5. Why might the parallel merge sort sometimes be slower than sequential for small arrays?
    6. Explain how load balancing is handled in this program.
    7. What is the difference between #pragma omp section and #pragma omp parallel sections?
    8. Why is thread synchronization important in merge sort?

2. Open MP – Static Scheduling with Chunk Size 2
    1. What does OMP_SCHEDULE static,2 mean?
    2. How does static scheduling differ from dynamic scheduling?
    3. In your program, how are iterations assigned to threads?
    4. If there are 4 threads and 8 iterations, how will the iterations be distributed?
    5. What environment variable is used to set schedule type in OpenMP?
    6. How do you retrieve the current thread ID in OpenMP?
    7. When would dynamic scheduling be better than static scheduling?
    8. What happens if chunk size is set to 1 instead of 2?

3. Open MP – Fibonacci Numbers Using Tasks
    1. What is the difference between pragma omp parallel for and #pragma omp task?
    2. How does task creation help in computing Fibonacci numbers?
    3. Why is Fibonacci a poor benchmark for parallel computing?
    4. What is the purpose of pragma omp taskwait?
    5. What is a "task dependency," and how can it be defined in OpenMP?
    6. What is the difference between shared and firstprivate in task variables?
    7. How can excessive task creation reduce performance?

4. OpenMP – Prime Numbers (Serial vs Parallel)
    1. How do you check if a number is prime in your program?
    2. Which OpenMP directive did you use to parallelize the loop?
    3. How do you handle race conditions in counting primes?
    4. What is the difference in execution time between serial and parallel versions? Why?
    5. How do you ensure that each thread gets a different range of numbers?
    6. What is false sharing, and could it occur in this program?
    7. Why might parallelization overhead outweigh benefits for small n?

5. MPI – Send and Receive
    1. What is the difference between MPI_Send and MPI_Recv?
    2. What are the parameters of MPI_Send? Explain each.

3. How do you avoid deadlock in MPI point-to-point communication?
4. What is the difference between blocking and non-blocking send?
5. What is the role of MPI_STATUS_IGNORE?
6. Can MPI_Send and MPI_Recv be used for broadcasting? Why or why not?

6. MPI – Deadlock Demonstration and Avoidance

1. How does altering the call sequence avoid deadlock?
2. Can non-blocking communication prevent deadlock? How?
3. What is the difference between synchronous and asynchronous sends?
4. Give an example of a situation that can cause MPI deadlock.
5. Why is it important to match MPI_Send and MPI_Recv calls correctly?

7.MPI – Broadcast

1. What is the purpose of MPI_Bcast?
2. How does broadcasting differ from point-to-point communication?
3. What is the root process in MPI_Bcast?
4. Can all processes be the root in broadcast?
5. How does MPI_Bcast improve performance compared to multiple sends?
6. What happens if processes in MPI_Bcast have different buffer sizes?

MPI – Scatter and Gather

What is the difference between MPI_Scatter and MPI_Gather?

7. How are data chunks distributed in MPI_Scatter?
8. What is the significance of the root process in these operations?
9. What happens if the number of elements is not divisible by the number of processes
10. Can MPI_Gather be used without a root process?
11. What is the difference between MPI_Gather and MPI_Allgather?

8.MPI – Reduce and Allreduce

12. What is the purpose of MPI_Reduce?
13. Explain the difference between MPI_Reduce and MPI_Allreduce.
14. What operations are supported by MPI reduction functions?
15. How does MPI_SUM work in reduction?
16. What is the advantage of using MPI_Allreduce over separate MPI_Reduce and MPI_Bcast?
17. What happens if different processes use different operations in reduction?